Day 15 of 15 · Capstone

A certificate you can check on the blockchain

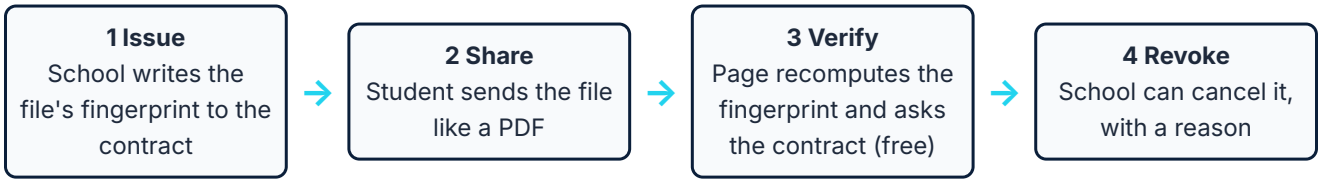
CertCheck: registry contract + web page + self-audit · 29 October 2026 · Phase 4 — Capstone · Manish Khadka

1. Learning objectives

- Combine hashing, signatures, access control, gas saving and a web front-end into one real application.
- Explain why only a **salted fingerprint** (hash) goes on-chain and never personal data.
- Design a verifier that asks "valid in *which* registry, on *which* chain, from *which* issuer?".
- Audit your own code like an outsider: find a bug, write the failing test, fix it, prove it.

2. The idea, step by step

Fake certificates are a real problem; checking one normally means emailing the school and waiting. CertCheck works like a **public notice board that nobody can secretly edit**.



Idea	Plain words
Fingerprint (keccak256 hash)	Same file → same code. One changed letter → a completely different code (Day 1).
Salt	16 random bytes inside the file. Without it, anyone could guess "Manish Khadka, Blockchain 101", hash it and confirm it. With it there are 2^{128} guesses per try.
Canonical JSON	One exact way to write the file (sorted keys, no spaces, normalised Unicode) so the same certificate always gives the same hash.
Per-issuer records	Each school has its own column `(issuer, hash)`, so one school can never take another's certificate (finding M1).
Registry + chain inside the file	A forger's copy of the contract is refused (finding M2).

Who can do what

	Issue	Revoke	Verify
Admin	only after making itself an issuer (visible)	any	yes
Issuer (school)	yes	its own, while still an issuer	yes
Anyone else	no	no	yes, free, no wallet

3. Key code, annotated

Storing a fingerprint (contract)

```
mapping(address issuer => mapping(bytes32 certHash => Record)) private _records;
struct Record { uint40 issuedAt; uint40 revokedAt; }   // 10 bytes = ONE storage slot

function _issue(bytes32 certHash) private {
    if (certHash == bytes32(0)) revert ZeroHash();
    Record storage r = _records[msg.sender][certHash]; // key includes the issuer (M1 fix)
    if (r.issuedAt != 0) revert AlreadyIssued(certHash); // a revoked record stays forever
    r.issuedAt = uint40(block.timestamp);
    emit CertificateIssued(msg.sender, certHash);
}
```

Revoking safely (finding M3)

```
bool ownActiveIssuer = msg.sender == issuer && hasRole(ISSUER_ROLE, msg.sender);
if (!ownActiveIssuer && !hasRole(DEFAULT_ADMIN_ROLE, msg.sender)) revert
NotAllowedToRevoke(msg.sender);
// a removed issuer (stolen key) can no longer cancel real certificates
```

Never lose the last admin (evening finding L2)

```
function _revokeRole(bytes32 role, address account) internal override returns (bool revoked) {
    if (role == DEFAULT_ADMIN_ROLE && hasRole(role, account) && adminCount == 1) revert
LastAdmin();
    revoked = super._revokeRole(role, account);
    if (revoked && role == DEFAULT_ADMIN_ROLE) --adminCount;
}
```

The verifier (off-chain)

```
cert = Certificate.parse(fileText);                // BAD_FILE if anything is odd
if (cert.registry != expected.address || cert.chainId != expected.chainId)
    return { verdict: "WRONG_REGISTRY" };           // M2
const r = await registry.verify(cert.issuer, Certificate.hash(cert)); // free read
// → VALID · VALID_OLD_ISSUER · REVOKED · NOT_FOUND
```

4. Results from the run

```
gas: deploy registry              755,992
gas: issue 1 certificate          70,391
gas: issue a class of 30 (batch)  766,567 (25,552 each)
gas: revoke                       56,087
verdict (genuine file):           VALID (free, a few milliseconds, no transaction)
verdict (one letter changed):     NOT_FOUND
verdict (after revoke):           REVOKED
non-issuer tries to issue:        rejected — AccessControlUnauthorizedAccount
bytes of personal data on-chain:  0
```

Batch issuing is about 2.8× cheaper per certificate than one by one (25,552 vs 70,391 gas).

5. QA & security findings

ID	Sev.	What went wrong	Fix
M1	Medium	First design stored certificates by hash only. A rival issuer could copy the pending transaction, pay more fee and become the "issuer".	Records per (issuer, hash)
M2	Medium	A forger deploys a copy of the contract and "issues" anything there.	File names registry + chain; verifier refuses others
M3	Medium	A removed issuer (stolen key) could still cancel real certificates.	Only an active issuer or the admin may revoke
L1	Low	Form default date used UTC: yesterday in Nepal before 05:45.	Use local date
L2	Low	**Evening:** the only admin could renounce itself → no admin ever again.	`adminCount` + `LastAdmin`

Tools: 46 tests passing (unit, fuzz, attacks, browser end-to-end, deploy, audit, exercises) · ESLint + Solhint clean · `npm audit`: 0 vulnerabilities · Slither: 3 triaged false positives (the `== 0` "never happened" markers). Known issues: one admin key (use a multisig), certificates issued by a thief before removal, address ≠ real school name, look-alike letters.

6. Common mistakes

- Putting names or grades on a public chain. Put a **salted hash** instead.
- Checking only "is this hash valid somewhere?" and not *registry, chain and issuer*.
- Forgetting that removing a role must remove **all** its powers.
- Letting one key be both owner and only admin. Plan the hand-over.
- Trusting a self-audit as a stamp. Get an independent review before real use.

7. Five interview questions

1. Why store a hash and not the certificate? Privacy and cost: the chain is public forever and storage is expensive. A hash proves the file existed unchanged without revealing it.

2. Why a salt? Hashes of guessable data can be brute-forced. 16 random bytes make each guess one of 2^{128}.

3. What is front-running and how did it hit v1? Anyone can read pending transactions and send a copy with a higher fee. In v1 a rival could take over a hash; per-issuer storage makes the copy harmless.

4. Why must the file contain the registry address and chain id? Otherwise an attacker deploys their own contract, issues the same hash there, and a naive verifier says "valid".

5. What is canonical JSON? A single agreed byte representation (sorted keys, no whitespace, Unicode NFC) so equal data always produces the same hash. See RFC 8785.

8. Exercises (tonight)

1. **Easy:** count how many of 256 fingerprint bits change after one letter (about half) and show reordered keys change 0.
2. **Medium:** turn a verdict into one plain sentence using a vetted issuer-name list (case-insensitive addresses; unknown issuer is flagged).
3. **Challenge:** verify a folder of files at once and flag two different files that reuse the same certificate id.

Cheat sheet
Hash = fingerprint · Salt = random bytes so guessing fails · Canonical = one exact text · Store per (issuer, hash) · Verify = registry + chain + issuer + status · Revoke = active issuer or admin · Never remove the last admin · Admin = multisig in production · Audit: failing test first, then the fix.

9. References

1. OpenZeppelin Contracts — Access Control: https://docs.openzeppelin.com/contracts/5.x/access-control
2. Solidity docs — Layout of state variables in storage: https://docs.soliditylang.org/en/v0.8.24/internals/layout_in_storage.html
3. Ethereum.org — Maximal extractable value (MEV): https://ethereum.org/en/developers/docs/mev/
4. RFC 8785 — JSON Canonicalization Scheme: https://www.rfc-editor.org/rfc/rfc8785
5. Blockcerts — open standard for blockchain credentials: https://www.blockcerts.org/guide/
6. Slither static analyzer: https://github.com/crytic/slither
7. Smart Contract Weakness Classification registry: https://swcregistry.io/
8. Consensys — Smart contract best practices: https://consensys.github.io/smart-contract-best-practices/
9. Antonopoulos & Wood, Mastering Ethereum: https://github.com/ethereumbook/ethereumbook